

CloudTrail, CloudWatch Task

Assignment Overview

This document covers six independent tasks: enabling CloudTrail with delivery to both S3 and CloudWatch Logs, adding SNS email alerts for CloudTrail activity, configuring CloudWatch monitoring for EC2 metrics such as CPU utilization, creating a CPU utilization alarm with email notification, and building two CloudWatch dashboards that monitor whether the Tomcat and Nginx services are running, each with its own alert. Each task resets and is documented with its own independent steps.

The following resource reference is used throughout this document:

Task	Resource	Purpose
1	CloudTrail trail + S3 bucket + CloudWatch Logs group	Capture and store API activity events
2	SNS topic + CloudTrail/EventBridge rule	Email alerts on CloudTrail events
3	CloudWatch Agent + EC2 metrics	Monitor CPU utilization and other EC2 metrics
4	CloudWatch Alarm (CPU > 70%)	Email alert on high CPU utilization
5	CloudWatch Dashboard + custom metric (Tomcat)	Alert if Tomcat service stops running
6	CloudWatch Dashboard + custom metric (Nginx)	Alert if Nginx service stops running

Task 1: Enable CloudTrail and Store Events in S3 and CloudWatch Logs

Outcome

A multi-region CloudTrail trail named org-cloudtrail is active, delivering all management events to an encrypted S3 bucket for long-term storage and streaming the same events to a CloudWatch Logs log group for near real-time search and alerting.

Objective

To capture a durable, queryable record of every API call made in the account, satisfying both audit/compliance retention and operational visibility needs.

Prerequisites

- IAM permissions to create CloudTrail trails, S3 buckets, and CloudWatch Logs resources.
- An IAM role that CloudTrail can assume to write into CloudWatch Logs.

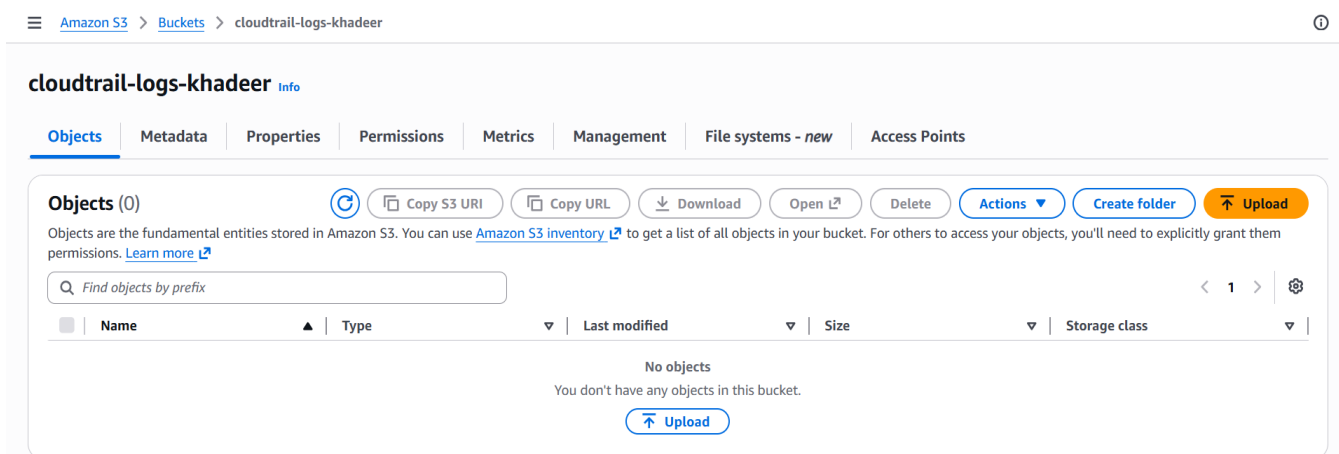
Step-by-Step Implementation

Step 1: Create the S3 Bucket for Trail Storage

S3 Console -> Create bucket -> configure:

Parameter	Value
Bucket name	cloudtrail-logs-khadeer
Region	us-east-1
Block Public Access	Enabled (all four settings)
Default encryption	SSE-S3 (or SSE-KMS)

Screenshot Evidence:

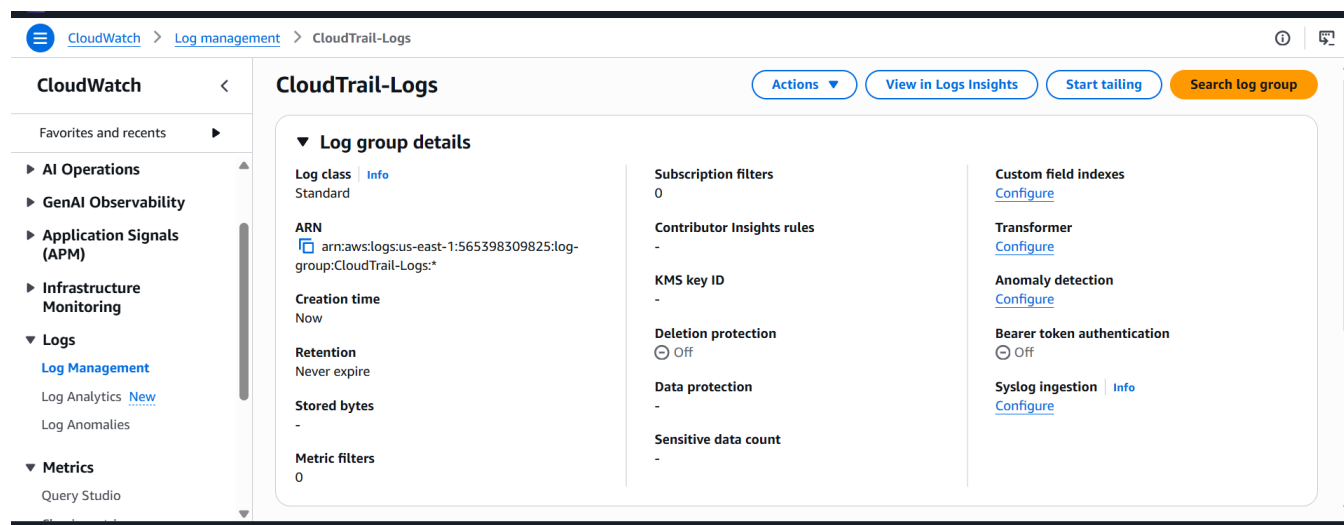


Step 2: Create the CloudWatch Logs Log Group and IAM Role

CloudWatch Console -> Log groups -> Create log group:

Parameter	Value
Log group name	CloudTrail-Logs
Retention	Never Expire
IAM role	CloudTrail_CloudWatchLogs_Role (auto-created via trail wizard)

Screenshot Evidence:



Step 3: Create the CloudTrail Trail

CloudTrail Console -> Trails -> Create trail -> configure:

Parameter	Value
Trail name	MyCloudTrail
Apply to all regions	Yes
S3 bucket	cloudtrail-logs-khadeer
CloudWatch Logs	Enabled
Log file validation	Enabled
Event type	Management events (Read + Write)

Screenshot Evidence:

Step 2

Choose log events

Step 3

Review and create

General details

A trail created in the console is a multi-region trail. [Learn more](#)

Trail name
Enter a display name for your trail.

3-128 characters. Only letters, numbers, periods, underscores, and dashes are allowed.

☐ Enable for all accounts in my organization
To review accounts in your organization, open AWS Organizations. [See all accounts](#)

Storage location | [Info](#)

☐ Create new S3 bucket
Create a bucket to store logs for the trail.

☒ Use existing S3 bucket
Choose an existing bucket to store logs for this trail.

Trail log bucket name
Enter a new S3 bucket name and folder (prefix) to store your logs. Bucket names must be globally unique.

Prefix - optional

CloudTrail > Trails > Create trail

CloudWatch Logs | [Info](#)

☒ Enabled

Log group | [Info](#)

☒ New

☐ Existing

Log group name

1-512 characters. Only letters, numbers, dashes, underscores, forward slashes, and periods are allowed.

IAM Role | [Info](#)

AWS CloudTrail assumes this role to send CloudTrail events to your CloudWatch Logs log group.

☒ New

☐ Existing

Role name

Policy document

CloudTrail > Trails > Create trail

Step 1

Choose trail attributes

Step 2

Choose log events

Step 3

Review and create

Choose log events

Events | [Info](#)
Record API activity for individual resources, or for all current and future resources in AWS account. [Additional charges apply](#)

Event type
Choose the type of events that you want to log.

☒ **Management events**
Capture management operations performed on your AWS resources.

☐ **Data events**
Log the resource operations performed on or within a resource.

☐ **Insights events**
Identify unusual activity, errors, or user behavior in your account.

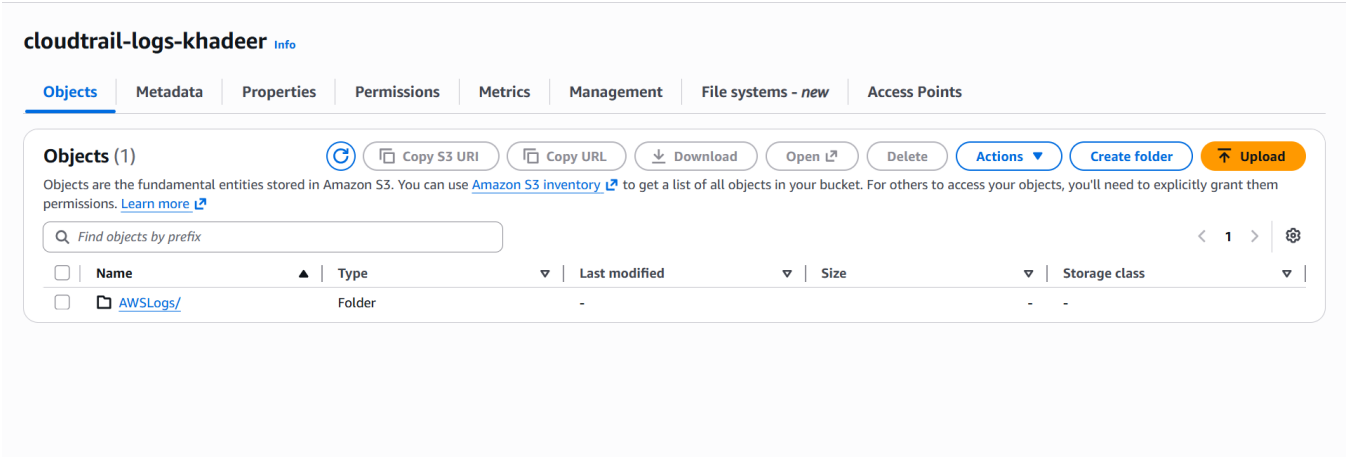
☐ **Network activity events**
Network activity events provide information about resource operations performed on a resource within a virtual private cloud endpoint.

Management events | [Info](#)

Validation Steps

- The trail status shows "Logging: On" in the CloudTrail console.
- New .json.gz objects appear under the S3 bucket's AWSLogs/ prefix within a few minutes of API activity.
- The same activity appears as log stream entries inside /aws/cloudtrail/ in CloudWatch Logs.

Screenshot Evidence:



Explanation

CloudTrail records every control-plane API call as an event. Sending events to S3 gives cheap, durable, long-term storage suited for audits and forensic review. Sending the same events to CloudWatch Logs in parallel makes them searchable with Logs Insights and usable as a trigger source for metric filters and alarms, which S3 storage alone cannot provide.

Real-Time Use Case

Security and compliance teams rely on the S3 copy for audits, evidence retention, and incident investigation, while operations teams use the CloudWatch Logs copy to build near real-time detections, such as alerting when a root user signs in or a security group is modified.

Common Errors and Troubleshooting

Error	Cause	Resolution
Trail created but CloudWatch Logs field stays empty	The trail's IAM role does not have permission to create log streams	Let the console auto-create the role, or attach the AWSCloudTrail_FullAccess-equivalent inline policy manually
No objects appear in S3	Bucket policy does not grant CloudTrail's service principal PutObject	Re-run the trail wizard so it attaches the required bucket policy automatically
Trail shows "Logging: Off"	Trail was created but not started	Select trail -> Start logging

Task 2: Enable SNS Alerts for CloudTrail Activity

Outcome

An SNS topic named cloudtrail-alerts is subscribed by email, and an EventBridge rule pushes selected CloudTrail events (for example, console sign-in or IAM changes) to that topic so the team receives an email within seconds of the event.

Objective

To turn passive log storage into an active notification path so specific high-value CloudTrail events reach a human inbox without anyone needing to query logs manually.

Prerequisites

- Task 1 completed (CloudTrail trail active and delivering to CloudWatch Logs).
- A valid email address to receive and confirm the SNS subscription.

Step-by-Step Implementation

Step 1: Create the SNS Topic

SNS Console -> Topics -> Create topic -> configure:

Parameter	Value
Type	Standard
Name	cloudtrail-alerts

Screenshot Evidence:

The first screenshot shows the 'Create topic' page in the AWS SNS console. The breadcrumb navigation is 'Amazon SNS > Topics > Create topic'. The 'Details' section shows the 'Type' as 'Standard' (selected) and 'Name' as 'cloudtrail-alerts'. The 'Display name' is optional and set to 'My Topic'. The second screenshot shows the 'cloudtrail-alerts' topic details page. The breadcrumb navigation is 'Amazon SNS > Topics > cloudtrail-alerts'. The 'Details' section shows the 'Name' as 'cloudtrail-alerts', 'ARN' as 'arn:aws:sns:us-east-1:565398309825:cloudtrail-alerts', 'Display name' as '-', and 'Type' as 'Standard'. The 'Topic owner' is '565398309825'. The 'Subscriptions' section shows 'Subscriptions (0)' and a 'Create subscription' button.

Step 2: Subscribe an Email Address

Topic -> Create subscription -> configure, then confirm via the link sent to the inbox:

Parameter	Value
Protocol	Email
Endpoint	Khadeershaik0329@gmail.com

Screenshot Evidence:

Amazon SNS > Subscriptions > Create subscription

Create subscription

Details

Topic ARN

arn:aws:sns:us-east-1:565398309825:cloudtrail-alerts

Protocol

The type of endpoint to subscribe

Email

Endpoint

An email address that can receive notifications from Amazon SNS.

khadeershaik0329@gmail.com

After your subscription is created, you must confirm it. [Info](#)



Simple Notification Service

Subscription confirmed!

You have successfully subscribed.

Your subscription's id is:

arn:aws:sns:us-east-1:565398309825:cloudtrail-alerts:3d622ae0-a05b-49a4-b4da-e35b33115651

If it was not your intention to subscribe, [click here to unsubscribe](#).

Step 3: Create an EventBridge Rule to Trigger the Topic

EventBridge Console -> Rules -> Create rule -> configure event pattern and target:

Parameter	Value
Rule name	cloudtrail-console-signin-rule
Event source	AWS Event or EventBridge partner events
Event pattern	use Pattern Form
Target	SNS topic: cloudtrail-alerts

Screenshot Evidence:

Amazon EventBridge

Dashboard

Developer resources

Learn
Sandbox
Quick starts

Buses

Event buses
Rules [Updated](#)
Global endpoints
Archives
Replays

Pipes

Pipes

Scheduler

- Step 1
Define rule detail
- Step 2
Build event pattern
- Step 3
Select target(s)
- Step 4 - optional
Configure tags
- Step 5
Review and create

Define rule detail [Info](#)

Rule detail

Name

Maximum of 64 characters consisting of numbers, lower/upper case letters, -, _.

Description - optional

Event bus [Info](#)

Select the event bus this rule applies to, either the default event bus or a custom or partner event bus.

☒ Enable the rule on the selected event bus

[Cancel](#)

[Next](#)

- Step 1
Define rule detail
- Step 2
Build event pattern
- Step 3
Select target(s)
- Step 4 - optional
Configure tags
- Step 5
Review and create

Build event pattern [Info](#)

Events

You don't have to select or enter a sample event, but it's recommended so you can reference it when writing and testing the event pattern, or filter criteria.

Event source

Select the event source from which events are sent.

☒ **AWS events or EventBridge partner events**

Events sent from AWS services or EventBridge partners.

☐ **Other**

Custom events or events sent from more than one source, e.g. events from AWS services and partners.

☐ **All events**

All events sent to your account.

► Sample event - optional

You don't have to select or enter a sample event, but it's recommended so you can reference it when writing and testing the event pattern, or filter criteria.

Event pattern [Info](#)

Creation method

☐ **Use schema**

Use an Amazon EventBridge schema to generate the event pattern.

☒ **Use pattern form**

Use a template provided by EventBridge to create an event pattern.

☐ **Custom pattern (JSON editor)**

Write an event pattern in JSON.

Event source

Event pattern

Target 1

Target types

Select an EventBridge event bus, EventBridge API destination (SaaS partner), or another AWS service as a target.

- ☐ EventBridge event bus
- ☐ EventBridge API destination
- ☒ AWS service

Select a target

[Info](#)

Select target(s) to invoke when an event matches your event pattern or when schedule is triggered (limit of 5 targets per rule)

SNS topic

Target location

☒ Target in this account

☐ Target in another AWS account

Topic

cloudtrail-alerts

Permissions

☒ Use execution role (recommended)

Execution role

EventBridge needs permission to send events to the target specified above. By continuing, you are allowing us to do so. [EventBridge and AWS Identity and Access Management](#)

☒ Create a new role for this specific resource

☐ Use existing role

Role name

Amazon_EventBridge_Invoke_Sns_1152618507

Validation Steps

- The email subscription shows status "Confirmed" in the SNS console.
- Triggering a matching event (for example signing in to the console) results in an email arriving at the subscribed address within roughly a minute.

Screenshot Evidence:

cloudtrail-console-signin-rule

[Edit](#)[Disable](#)[Delete](#)[CloudFormation Template](#)

Rule details

[Info](#)

Rule name

cloudtrail-console-signin-rule

Description

-

Status

Enabled

Rule ARN

arn:aws:events:us-east-1:565398309825:rule/cloudtrail-console-signin-rule

Event bus name

default

Event bus ARN

arn:aws:events:us-east-1:565398309825:event-bus/default

Type

Standard

Event pattern

[Targets](#)[Monitoring](#)[Tags](#)

Event pattern

[Info](#)[Edit](#)

```
1 {
2   "source": ["aws.cloudtrail"],
3   "detail-type": ["AWS API Call via CloudTrail"],
4   "detail": {
5     "eventSource": ["cloudtrail.amazonaws.com"]
6   }
7 }
```

[Copy](#)

Explanation

EventBridge continuously receives CloudTrail's event stream and matches it against rule patterns; a match publishes a message to SNS, which fans it out to every confirmed subscriber. This decouples detection (the rule)

from notification (the subscriber list), so adding a Slack or SMS subscriber later needs no change to the detection logic.

Real-Time Use Case

Security teams commonly alert on root account usage, IAM policy changes, or security group modifications this way, giving an on-call engineer immediate notice of a sensitive change instead of discovering it during a periodic log review.

Common Errors and Troubleshooting

Error	Cause	Resolution
Subscription stuck "Pending confirmation"	Confirmation link in the email was not clicked	Open the email and click Confirm subscription
No email received for a matching event	Event pattern does not match the actual event field names/values	Test the pattern in the EventBridge rule's built-in event pattern tester
Too many unrelated emails	Event pattern is too broad (matches all CloudTrail events)	Narrow the pattern to specific eventName / eventSource values

Task 3: Configure CloudWatch Monitoring for EC2 Metrics

Outcome

The CloudWatch Agent is installed and running on the target EC2 instance, publishing CPU utilization plus memory and disk metrics (which are not available by default) into a custom namespace alongside the standard EC2 metrics already collected automatically.

Objective

To get a complete, instance-level view of resource utilization beyond the basic host-level metrics AWS provides out of the box.

Prerequisites

- An EC2 instance with an IAM role attached that includes the CloudWatchAgentServerPolicy.
- SSH or Session Manager access to the instance.

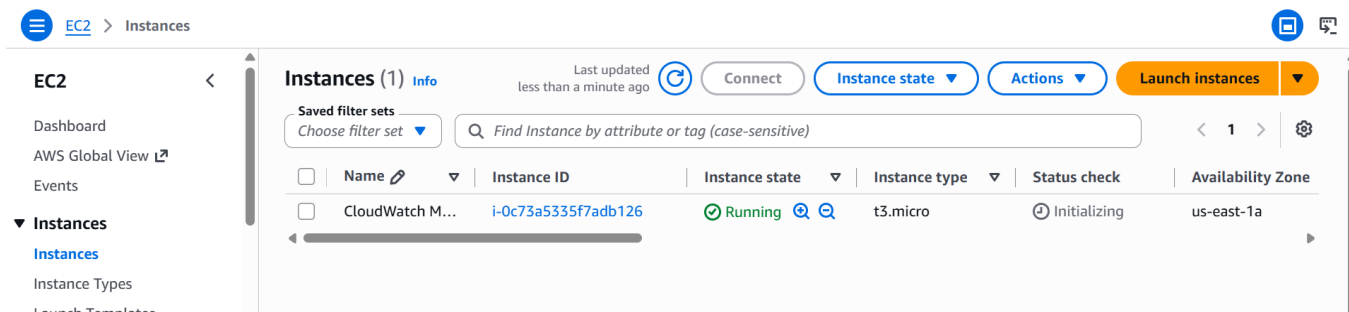
Step-by-Step Implementation

Step 1: Attach the IAM Role and Install the Agent

EC2 Console -> select instance -> Actions -> Security -> Modify IAM role -> attach CloudWatchAgentServerPolicy. Then on the instance:

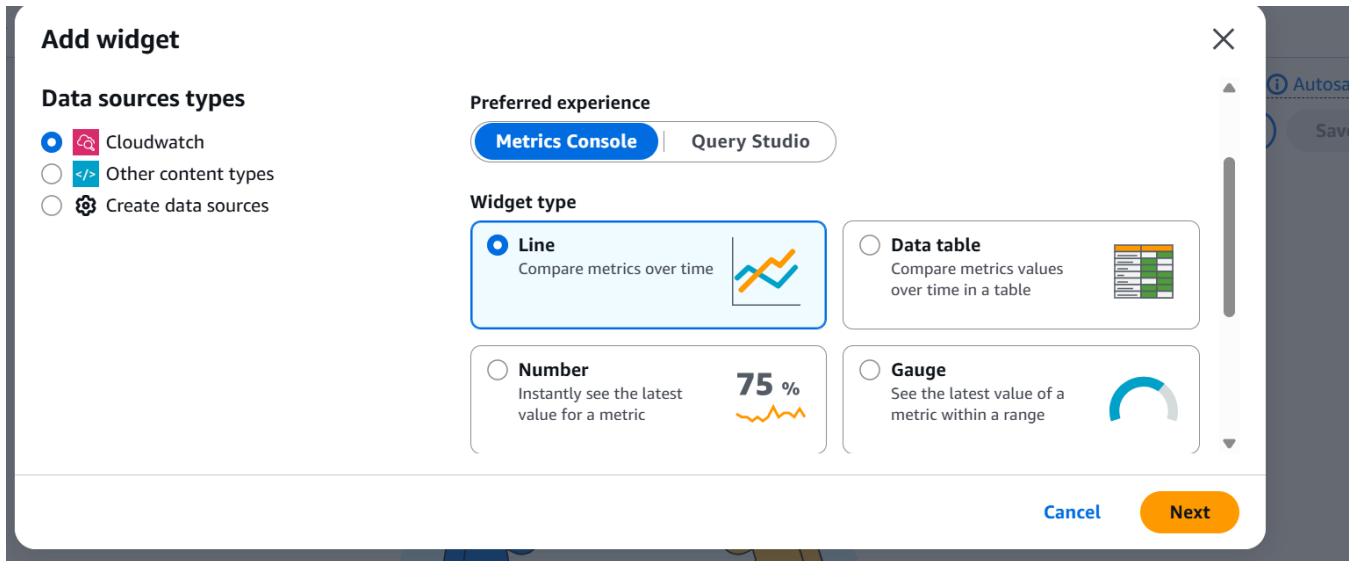
Parameter	Value
Install command	<code>sudo yum install amazon-cloudwatch-agent -y</code>
Config wizard	<code>sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-config-wizard</code>

Screenshot Evidence: capture the console view for this step and insert it here.



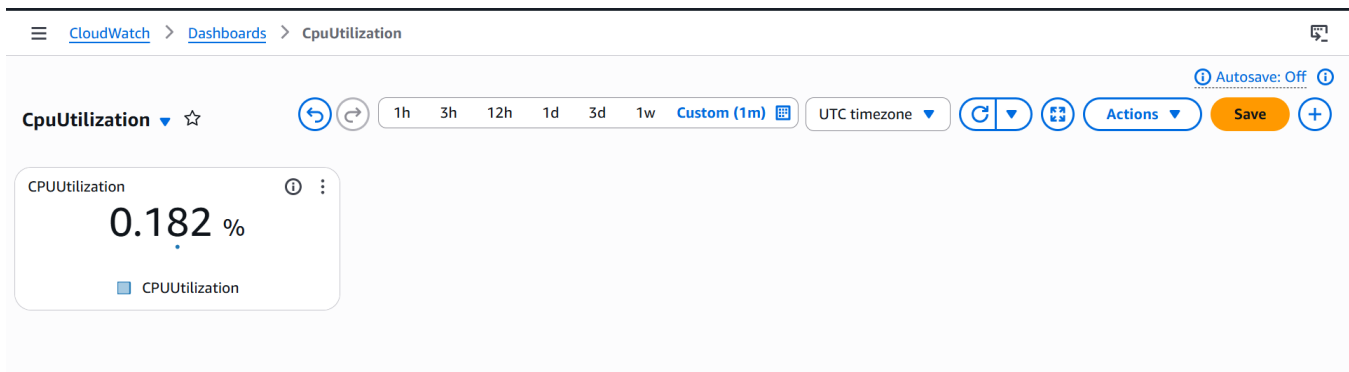
Step 2: Open cloud watch and Create dashboard

Screenshot Evidence: capture the console view for this step and insert it here.



Step 3: select Ec2 and then per metric give InstanceId , select cpu utilization

Screenshot Evidence:



Validation Steps

- Values update roughly every 60 seconds and change when load is generated on the instance.

Explanation

EC2 automatically reports basic host-level metrics such as CPUUtilization at 5-minute resolution with no agent required, but memory and disk usage are only visible from inside the guest OS. The CloudWatch Agent runs on

the instance, reads OS-level counters, and pushes them to CloudWatch as custom metrics, closing that visibility gap.

Real-Time Use Case

Application teams use these combined metrics to catch memory leaks or disk-space exhaustion that basic EC2 monitoring would miss, well before those conditions cause an outage.

Task 4: Create a CPU Utilization Alarm with Email Notification

Outcome

A CloudWatch alarm named `ec2-high-cpu-alarm` enters ALARM state whenever average CPU utilization exceeds 70 percent for two consecutive 5-minute periods, and publishes to the `cloudtrail-alerts` SNS topic so an email is sent immediately.

Objective

To get proactive notice of sustained high CPU load before it degrades application performance, instead of discovering it after users complain.

Prerequisites

- Task 3 completed, or the standard EC2 CPUUtilization metric available.
- An existing SNS topic with a confirmed email subscription (from Task 2, or a new topic created the same way).

Step-by-Step Implementation

Step 1: Open the Alarm Creation Wizard

CloudWatch Console -> Alarms -> Create alarm -> Select metric -> EC2 -> Per-Instance Metrics -> CPUUtilization.

Screenshot Evidence:

The screenshot shows the AWS CloudWatch 'Create alarm' wizard, specifically Step 1: 'Specify metric and conditions'. The breadcrumb navigation at the top reads 'CloudWatch > Alarms > Create alarm'. On the left, a vertical list of steps shows 'Step 1: Specify metric and conditions' as the active step, followed by 'Step 2: Configure actions', 'Step 3: Add alarm details', and 'Step 4: Preview and create'. The main content area is titled 'Specify metric and conditions' and includes a toggle for 'Alarm recommendations' (which is turned on) and a 'View details' link. Below this, there are two sections: 'Data source' and 'Type'. The 'Data source' section, titled 'Select the type of data to monitor', has three options: 'Metrics' (selected with a blue radio button and described as 'Monitor CloudWatch metrics'), 'Logs' (described as 'Monitor CloudWatch log queries'), and 'Alarms' (described as 'Combine multiple alarms'). The 'Type' section, titled 'Choose the metric type for this alarm.', has two options: 'Classic' (selected with a blue radio button and described as 'Use the standard CloudWatch metric selection experience.') and 'PromQL' (described as 'Use PromQL queries to define the metric for this alarm.').

Next

Step 1

Specify metric and conditions

Step 2

Configure actions

Step 3

Add alarm details

Step 4

Preview and create

Configure actions

Notification

Alarm state trigger
Define the alarm state that will trigger this action.

☒ **In alarm**
The metric or expression is outside of the defined threshold.

☐ **OK**
The metric or expression is within the defined threshold.

☐ **Insufficient data**
The alarm has just started or not enough data is available.

Remove

Send a notification to the following SNS topic
Define the SNS (Simple Notification Service) topic that will receive the notification.

☒ **Select an existing SNS topic**

☐ Create new topic

☐ Use topic ARN to notify other accounts

Send a notification to...

Only topics belonging to this account are listed here. All persons and applications subscribed to the selected topic will receive notifications.

Email (endpoints)
khadeershaik0329@gmail.com - [View in SNS Console](#)

Add notification

Step 3: Attach the Notification Action

Configure actions -> Alarm state trigger:

Parameter	Value
Alarm name	AWS/EC2 CPUUtilization InstanceId=i0415f4ed30699aacd
Notification	cloudtrail-alerts (existing SNS topic)
Trigger	In alarm

Screenshot Evidence:

CloudWatch > Alarms

CloudWatch

Alarms

Mute rules - new

Alarms (1)

Filter alarms

Quick filters

Clear selection

Create composite alarm

Actions

Create alarm

	Name	State	Actions	Actions muted - new	Last state update (UTC)	Conditions
☐	AWS/EC2 CPUUtilization InstanceId=i- 0415f4ed30699aacd	OK	Actions enabled	-	2026-07-28 09:58:05	CPUUtilization > 80 for 3 datapoints \ minutes

1 of 757

ALARM: "AWS/EC2 CPUUtilization InstanceId=i-0415f4ed30699aacd" in US East (N. Virginia)

Inbox x

AWS Notifications

<no-reply@sns.amazonaws.com>

15:38 (3 minutes ago)

to me

You are receiving this email because your Amazon CloudWatch Alarm "AWS/EC2 CPUUtilization InstanceId=i-0415f4ed30699aacd" in the US East (N. Virginia) region has entered the ALARM state, because "Threshold Crossed: 3 out of the last 3 datapoints [99.99500482866563 (28/07/26 10:07:00), 99.04904904904905 (28/07/26 10:06:00), 99.59834002766621 (28/07/26 10:05:00)] were greater than the threshold (80.0) (minimum 3 datapoints for OK -> ALARM transition)." at "Tuesday 28 July, 2026 10:08:05 UTC".

View this alarm in the AWS Management Console:
<https://us-east-1.console.aws.amazon.com/cloudwatch/deeplink.js?region=us-east-1#alarmsV2:alarm/AWS%2FEC2%20CPUUtilization%20InstanceId%3Di-0415f4ed30699aacd>

Alarm Details:

- Name: AWS/EC2 CPUUtilization InstanceId=i-0415f4ed30699aacd
- Description: This alarm helps to monitor the CPU utilization of an EC2 instance. Depending on the application, consistently high utilization levels might be normal. But if performance is degraded, and the application is not constrained by disk I/O, memory, or network resources, then a

Reply

Forward

Validation Steps

- The alarm shows state "OK" under normal load in the CloudWatch Alarms console.
- Generating sustained CPU load (for example with a stress-test tool) moves the alarm to "In alarm" and an email arrives at the subscribed address within a few minutes.

Explanation

The alarm evaluates the CPUUtilization metric on a rolling window and compares the average against the 70 percent threshold. Requiring 2 consecutive breaching datapoints avoids false alarms from a single brief spike, while still catching genuinely sustained load quickly enough to act on it.

Real-Time Use Case

This is a standard first-line defense for any production instance: it gives operations a heads-up to investigate or scale before CPU saturation causes request timeouts or dropped connections.

Common Errors and Troubleshooting

Error	Cause	Resolution
Alarm stuck in "Insufficient data"	Instance is not actively reporting the metric, or period too short after launch	Wait for a full metric period to elapse, confirm the instance is running
Alarm never triggers under real load	Datapoints-to-alarm set too high, or period too long	Lower datapoints required or shorten the period
No email received when alarm fires	SNS subscription not confirmed	Reconfirm the subscription from the original confirmation email

Task 5: Create a Dashboard to Monitor the Tomcat Service

Outcome

A CloudWatch dashboard named service-health-dashboard displays a widget showing whether the Tomcat process is running on the EC2 instance, and a CloudWatch alarm sends an email the moment Tomcat stops.

Objective

To detect and alert on application-level failure (the service itself crashing or being stopped) rather than only infrastructure-level failure, since a healthy EC2 instance can still be serving zero traffic if Tomcat has died.

Prerequisites

- CloudWatch Agent installed (Task 3) with the procstat plugin available.
- Tomcat installed and running on the instance.

Step-by-Step Implementation

Step 1: install java

Yum install install java-21-amazon-corretto java-21-amazon-corretto-devel

Screenshot:

```
[ec2-user@ip-172-31-22-74 ~]$ sudo -i
[root@ip-172-31-22-74 ~]# yum install java-21-amazon-corretto java-21-amazon-corretto-devel
Last metadata expiration check: 0:01:02 ago on Tue Jul 28 11:54:58 2026.
Dependencies resolved.
=====
Package                                Architecture      Version           Repository        Size
=====
Installing:
java-21-amazon-corretto               x86_64            1:21.0.11+10-1.amzn2023.1  amazonlinux      218 k
java-21-amazon-corretto-devel         x86_64            1:21.0.11+10-1.amzn2023.1  amazonlinux      149 k
Installing dependencies:
alsa-lib                              x86_64            1.2.7.2-1.amzn2023.0.3     amazonlinux      503 k
cairo                                  x86_64            1.18.0-4.amzn2023.0.3     amazonlinux      717 k
dejavu-sans-fonts                     noarch            2.37-16.amzn2023.0.2       amazonlinux      1.3 M
dejavu-sans-mono-fonts                noarch            2.37-16.amzn2023.0.2       amazonlinux      467 k
dejavu-serif-fonts                   noarch            2.37-16.amzn2023.0.2       amazonlinux      1.0 M
fontconfig                            x86_64            2.13.94-2.amzn2023.0.2     amazonlinux      273 k
fonts-filessystem                     noarch            1:2.0.5-12.amzn2023.0.2     amazonlinux      9.5 k
freetype                              x86_64            2.13.2-5.amzn2023.0.2     amazonlinux      422 k
giflib                                x86_64            5.2.1-9.amzn2023.0.4       amazonlinux      49 k
google-noto-fonts-common              noarch            20240401-1.amzn2023.0.2     amazonlinux      17 k
google-noto-sans-vf-fonts             noarch            20240401-1.amzn2023.0.2     amazonlinux      593 k
graphite2                              x86_64            1.3.14-7.amzn2023.0.3     amazonlinux      97 k
harfbuzz                              x86_64            7.0.0-2.amzn2023.0.2       amazonlinux      873 k
java-21-amazon-corretto-headless      x86_64            1:21.0.11+10-1.amzn2023.1  amazonlinux      97 M
javapackages-filessystem              noarch            6.0.0-7.amzn2023.0.6       amazonlinux      12 k
langpacks-core-font-en                noarch            3.0-21.amzn2023.0.4       amazonlinux      10 k
libICE                                x86_64            1.1.1-3.amzn2023.0.1       amazonlinux      76 k
libSM                                  x86_64            1.2.4-3.amzn2023.0.1       amazonlinux      45 k

Verifying      : pixman-0.43.4-1.amzn2023.0.4.x86_64
Verifying      : xml-common-0.6.3-56.amzn2023.0.2.noarch

Installed:
alsa-lib-1.2.7.2-1.amzn2023.0.3.x86_64                cairo-1.18.0-4.amzn2023.0.3.x86_64
dejavu-sans-fonts-2.37-16.amzn2023.0.2.noarch          dejavu-sans-mono-fonts-2.37-16.amzn2023.0.2.noarch
dejavu-serif-fonts-2.37-16.amzn2023.0.2.noarch         fontconfig-2.13.94-2.amzn2023.0.2.x86_64
fonts-filessystem-1:2.0.5-12.amzn2023.0.2.noarch       freetype-2.13.2-5.amzn2023.0.2.x86_64
giflib-5.2.1-9.amzn2023.0.4.x86_64                   google-noto-fonts-common-20240401-1.amzn2023.0.2.noarch
google-noto-sans-vf-fonts-20240401-1.amzn2023.0.2.noarch
harfbuzz-7.0.0-2.amzn2023.0.2.x86_64                 java-21-amazon-corretto-1:21.0.11+10-1.amzn2023.1.x86_64
java-21-amazon-corretto-devel-1:21.0.11+10-1.amzn2023.1.x86_64
javapackages-filessystem-6.0.0-7.amzn2023.0.6.noarch   java-21-amazon-corretto-headless-1:21.0.11+10-1.amzn2023.1.x86_64
langpacks-core-font-en-3.0-21.amzn2023.0.4.noarch     libSM-1.2.4-3.amzn2023.0.1.x86_64
libICE-1.1.1-3.amzn2023.0.1.x86_64                   libX11-1.8.10-2.amzn2023.0.1.x86_64
libX11-1.8.10-2.amzn2023.0.1.x86_64                  libX11-common-1.8.10-2.amzn2023.0.1.noarch
libXau-1.0.11-6.amzn2023.0.1.x86_64                  libXext-1.3.6-1.amzn2023.0.1.x86_64
libXi-1.8.2-1.amzn2023.0.1.x86_64                    libXinerama-1.1.5-6.amzn2023.0.1.x86_64
libXrandr-1.5.4-3.amzn2023.0.1.x86_64                 libXrender-0.9.11-6.amzn2023.0.1.x86_64
libXt-1.3.0-3.amzn2023.0.1.x86_64                   libXtst-1.2.5-1.amzn2023.0.1.x86_64
libbrotli-1.0.9-4.amzn2023.0.2.x86_64                libjpeg-turbo-2.1.4-2.amzn2023.0.5.x86_64
libpng-2:1.6.37-10.amzn2023.0.13.x86_64              libxcb-1.17.0-1.amzn2023.0.1.x86_64
pixman-0.43.4-1.amzn2023.0.4.x86_64                  xml-common-0.6.3-56.amzn2023.0.2.noarch

Complete!
```

Step 2: install Apache tomcat and run Apache tomcat

Wget https://dlcdn.apache.org/tomcat/tomcat-10/v10.1.57/bin/apache-tomcat-10.1.57.tar.gz

Screenshot Evidence:

```

-rw-r-----. 1 root root 27078 Jul 3 13:06 commons-daemon.jar
-rw-r-----. 1 root root 2040 Jul 3 13:06 configtest.bat
-rwxr-x---. 1 root root 1922 Jul 3 13:06 configtest.sh
-rwxr-x---. 1 root root 8319 Jul 3 13:06 daemon.sh
-rw-r-----. 1 root root 2091 Jul 3 13:06 digest.bat
-rwxr-x---. 1 root root 1965 Jul 3 13:06 digest.sh
-rw-r-----. 1 root root 3606 Jul 3 13:06 makebase.bat
-rwxr-x---. 1 root root 3382 Jul 3 13:06 makebase.sh
-rw-r-----. 1 root root 2096 Jul 3 13:06 migrate.bat
-rwxr-x---. 1 root root 1970 Jul 3 13:06 migrate.sh
-rw-r-----. 1 root root 3382 Jul 3 13:06 setclasspath.bat
-rwxr-x---. 1 root root 3941 Jul 3 13:06 setclasspath.sh
-rw-r-----. 1 root root 2020 Jul 3 13:06 shutdown.bat
-rwxr-x---. 1 root root 1902 Jul 3 13:06 shutdown.sh
-rw-r-----. 1 root root 2022 Jul 3 13:06 startup.bat
-rwxr-x---. 1 root root 1904 Jul 3 13:06 startup.sh
-rw-r-----. 1 root root 52517 Jul 3 13:06 tomcat-juli.jar
-rw-r-----. 1 root root 476797 Jul 3 13:06 tomcat-native.tar.gz
-rw-r-----. 1 root root 3623 Jul 3 13:06 tool-wrapper.bat
-rwxr-x---. 1 root root 4600 Jul 3 13:06 tool-wrapper.sh
-rw-r-----. 1 root root 2026 Jul 3 13:06 version.bat
-rwxr-x---. 1 root root 1908 Jul 3 13:06 version.sh
[root@ip-172-31-22-74 bin]# ./startup.sh
Using CATALINA_BASE:   /root/tomcat10
Using CATALINA_HOME:   /root/tomcat10
Using CATALINA_TMPDIR: /root/tomcat10/temp
Using JRE_HOME:        /usr
Using CLASSPATH:       /root/tomcat10/bin/bootstrap.jar:/root/tomcat10/bin/tomcat-juli.jar
Using CATALINA_OPTS:
Tomcat started.
[root@ip-172-31-22-74 bin]#

```

← → ↻ 🔒 Not secure 3.94.132.156:8080 ☆ 🔴 📄 🔑 ⋮

Home Documentation Configuration Examples Wiki Mailing Lists Find Help

Apache Tomcat/10.1.57

If you're seeing this, you've successfully installed Tomcat. Congratulations!



Recommended Reading:

- [Security Considerations How-To](#)
- [Manager Application How-To](#)
- [Clustering/Session Replication How-To](#)

[Server Status](#) [Manager App](#) [Host Manager](#)

Developer Quick Start

- [Tomcat Setup](#)
- [First Web Application](#)
- [Realms & AAA](#)
- [JDBC DataSources](#)
- [Examples](#)
- [Servlet Specifications](#)
- [Tomcat Versions](#)

Managing Tomcat

For security, access to the [manager webapp](#) is restricted. Users are defined in:

```
$CATALINA_HOME/conf/tomcat-users.xml
```

In Tomcat 10.1 access to the manager

Documentation

- [Tomcat 10.1 Documentation](#)
- [Tomcat 10.1 Configuration](#)
- [Tomcat Wiki](#)

Getting Help

[FAQ and Mailing Lists](#)

The following mailing lists are available:

- [tomcat-announce](#)

Important announcements, releases, security

Step 3: create Script file and Executed the Script

- Vi monitor.sh -> ./monitor.sh

Screenshot Evidence:

```

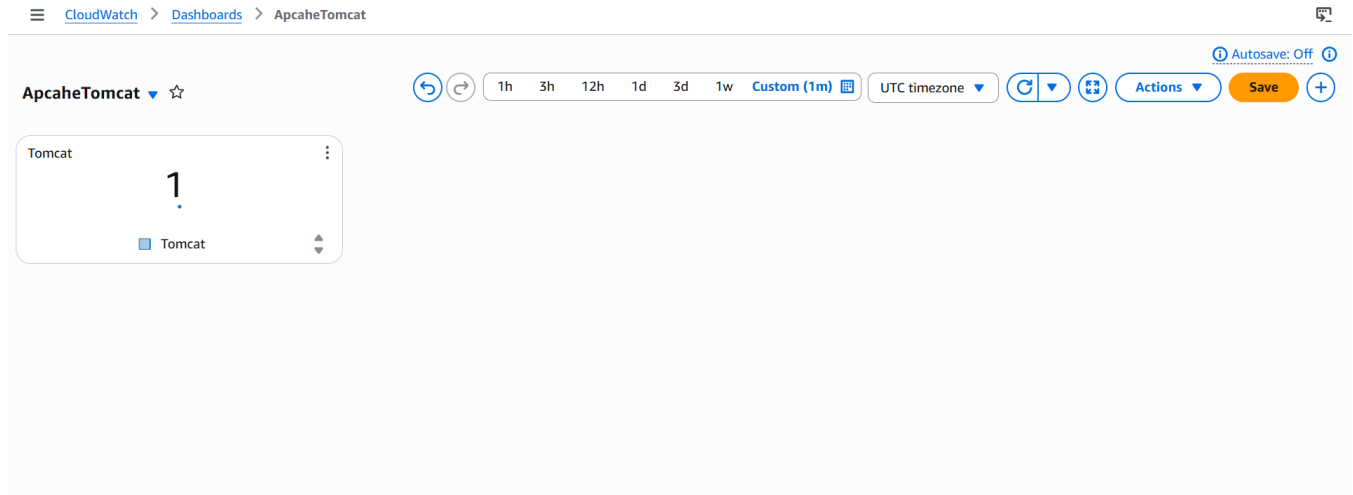
[root@ip-172-31-22-74 ~]# vi monitor.sh
[root@ip-172-31-22-74 ~]# chmod +x monitor.sh
[root@ip-172-31-22-74 ~]# ./monitor.sh

Unable to locate credentials. You can configure credentials by running "aws login".
[root@ip-172-31-22-74 ~]# aws configure
AWS Access Key ID [None]: AKIAIYHJC5RPA55YPHKCN
AWS Secret Access Key [None]: 2saXcTlWSN0wETur4rpKsALZ2ywh7TorKUAGokbe
Default region name [None]: us-east-1
Default output format [None]: json
[root@ip-172-31-22-74 ~]# ./monitor.sh
[root@ip-172-31-22-74 ~]#

```

Step 4: create a Dashboard for Tomcat

Dashboard -> create dashboard -> tomcat



Explanation

procstat is a CloudWatch Agent plugin that counts how many OS processes match a given pattern or PID file and reports that count as a metric. Because it runs entirely from the OS process table, it detects a crashed or manually stopped service immediately, independent of whether the underlying EC2 instance itself is still healthy.

Real-Time Use Case

Teams running Java application servers use this pattern to catch scenarios like an out-of-memory crash or an accidental sudo systemctl stop, which would otherwise go unnoticed until a user reports the site is down.

Common Errors and Troubleshooting

Error	Cause	Resolution
procstat metric never appears	Pattern or pid_file does not match the actual running process	Verify with <code>ps aux grep tomcat</code> and correct the pattern/path
Metric flaps between 0 and 1 during startup	Alarm evaluated before Tomcat fully finished starting	Increase the alarm's evaluation period or datapoints required
Alarm fires but Tomcat is actually running	PID file is stale from a previous restart	Point procstat at the process name pattern instead of the PID file

Task 6: Create a Dashboard to Monitor the Nginx Service

Outcome

The same service-health-dashboard gains a second widget showing whether Nginx is running, and a companion alarm nginx-down-alarm sends an email the moment the Nginx process count drops to zero.

Objective

To extend application-level health monitoring to the web/reverse-proxy tier, so a stopped Nginx process is caught with the same speed and reliability as a stopped Tomcat process.

Prerequisites

- CloudWatch Agent installed (Task 3) with the procstat plugin available.
- Nginx installed and running on the instance.

Step-by-Step Implementation

Step 1: install nginx and run Nginx

Yum install nginx

Systemctl start nginx

Syatemctl enable nginx

Screenshot Evidence:

```
root@ip-172-31-22-74 ~]# sudo yum update -y
sudo yum install nginx -y
Last metadata expiration check: 1:54:16 ago on Tue Jul 28 11:54:58 2026.
Dependencies resolved.
Nothing to do.
Complete!
Last metadata expiration check: 1:54:18 ago on Tue Jul 28 11:54:58 2026.
Dependencies resolved.
=====
Package                                Architecture      Version            Repository          Size
=====
Installing:
nginx                                  x86_64            1:1.30.3-1.amzn2023.0.1  amazonlinux        34 k
Installing dependencies:
generic-logos-httpd                   noarch            18.0.0-12.amzn2023.0.3  amazonlinux        19 k
gperftools-libs                       x86_64            2.9.1-1.amzn2023.0.3    amazonlinux        308 k
libunwind                             x86_64            1.4.0-5.amzn2023.0.3    amazonlinux        66 k
nginx-core                            x86_64            1:1.30.3-1.amzn2023.0.1  amazonlinux        709 k
nginxfilesystem                       noarch            1:1.30.3-1.amzn2023.0.1  amazonlinux        10 k
nginx-mimetypes                       noarch            2.1.49-3.amzn2023.0.3   amazonlinux        21 k
=====
Transaction Summary
=====
Installing:
  nginx-1:1.30.3-1.amzn2023.0.1.x86_64
Installing dependencies:
  generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch
  gperftools-libs-2.9.1-1.amzn2023.0.3.x86_64
  libunwind-1.4.0-5.amzn2023.0.3.x86_64
  nginx-core-1:1.30.3-1.amzn2023.0.1.x86_64
  nginxfilesystem-1:1.30.3-1.amzn2023.0.1.noarch
  nginx-mimetypes-2.1.49-3.amzn2023.0.3.noarch
Total download size: 1.1 MB
Installed size: 4.1 MB
Is this ok [y/N]: y
(1/1) nginx-core-1:1.30.3-1.amzn2023.0.1.x86_64.rpm: [OK]
(2/1) nginxfilesystem-1:1.30.3-1.amzn2023.0.1.noarch.rpm: [OK]
(3/1) nginx-mimetypes-2.1.49-3.amzn2023.0.3.noarch.rpm: [OK]
(4/1) gperftools-libs-2.9.1-1.amzn2023.0.3.x86_64.rpm: [OK]
(5/1) libunwind-1.4.0-5.amzn2023.0.3.x86_64.rpm: [OK]
(6/1) generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch.rpm: [OK]
(7/1) nginx-1:1.30.3-1.amzn2023.0.1.x86_64.rpm: [OK]
Complete!
[root@ip-172-31-22-74 ~]# sudo systemctl start nginx
[root@ip-172-31-22-74 ~]# systemctl start nginx
[root@ip-172-31-22-74 ~]# sudo systemctl start nginx
[root@ip-172-31-22-74 ~]# sudo systemctl enable nginx
Created symlink /etc/systemd/system/multi-user.target.wants/nginx.service -> /usr/lib/systemd/system/nginx.service.
[root@ip-172-31-22-74 ~]# sudo systemctl status nginx
• nginx.service - The nginx HTTP and reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: disabled)
   Active: active (running) since Tue 2026-07-28 13:49:30 UTC; 43s ago
 Main PID: 2902 (nginx)
   Tasks: 3 (limit: 1059)
  Memory: 3.4M
    CPU: 57ms
  CGroup: /system.slice/nginx.service
          └─2902 "nginx: master process /usr/sbin/nginx"
```

Step 3: create Script file and Exceuted the Script

- Vi monitor.sh -> ./monitor.sh

Screenshot Evidence:

```
#!/bin/bash

REGION="us-east-1"
NAMESPACE="Nginx"
METRIC_NAME="Nginx"

# Get EC2 Instance ID
INSTANCE_ID=$(/usr/bin/ec2-metadata -i | awk '{print $2}')

# Check Nginx status
if systemctl is-active --quiet nginx
then
    VALUE=1
else
    VALUE=0
fi

# Send metric to CloudWatch
/usr/bin/aws cloudwatch put-metric-data \
    --region "$REGION" \
    --namespace "$NAMESPACE" \
    --metric-name "$METRIC_NAME" \
    --value "$VALUE" \
    --dimensions InstanceId="$INSTANCE_ID"

~
"nginx.sh" 24L, 476B
```

Step 4: create a Dashboard for nginx

Dashboard -> create dashboard -> nginx

Screenshot Evidence:



Because each procstat entry watches an independent process pattern, Tomcat and Nginx are monitored and alarmed on separately even though they share the same agent and the same dashboard. This means a failure in one service does not mask or get confused with the health of the other.

Real-Time Use Case

In a typical stack where Nginx reverse-proxies to Tomcat, monitoring both independently lets on-call staff immediately tell whether an outage is a proxy failure, an application-server failure, or both, cutting diagnosis time during an incident.

Common Errors and Troubleshooting

Error	Cause	Resolution
Nginx widget shows no data while Tomcat's works	Nginx procstat block missing or misconfigured in the same JSON file	Re-check the config file syntax; each procstat entry is a separate array item

Error	Cause	Resolution
Both alarms fire together on reboot	Expected transient behavior while services restart	Add a short delay/datapoints-to-alarm buffer to absorb normal restart time
Alert email doesn't specify which service is down	Default SNS message uses the generic alarm description	Set a distinct, descriptive AlarmDescription for each alarm